LOW-LEVEL DESIGN (LLD)

RAG-Based Customer Support Assistant

Using LangGraph and Human-in-the-Loop (HITL)

1. Module-Level Design

The Low-Level Design defines how individual modules of the RAG-Based Customer Support Assistant are implemented internally. Each module is responsible for performing a specific function within the system.

The system is divided into multiple functional modules to ensure modularity, maintainability, and scalability.

1.1 Document Processing Module

The Document Processing Module is responsible for loading PDF documents and extracting textual content.

This module reads the input knowledge base file and converts the content into machine-readable text. The extracted text is passed to the chunking module for further processing.

Primary Responsibilities:

Load PDF files

Extract text from each page

Clean and preprocess text

Send extracted text to the chunking module

Tools Used:

PyPDF library

1.2 Chunking Module

The Chunking Module divides large text content into smaller segments called chunks.

Chunking is necessary because large documents cannot be processed efficiently as a single unit. Smaller chunks allow faster embedding generation and improve retrieval accuracy.

Primary Responsibilities:

Split text into smaller segments

Maintain logical sentence boundaries

Assign unique identifiers to each chunk

Tools Used:

Recursive Text Splitter

1.3 Embedding Module

The Embedding Module converts text chunks into numerical vector representations.

These vectors capture the semantic meaning of text and allow similarity-based searching. Each chunk is transformed into a high-dimensional embedding.

Primary Responsibilities:

Generate embeddings from text chunks

Store embedding metadata

Pass embeddings to the vector storage module

Tools Used:

Sentence Transformers

1.4 Vector Storage Module

The Vector Storage Module stores embeddings in a structured format.

This module enables efficient retrieval of relevant chunks during query processing. Embeddings are indexed to support similarity search.

Primary Responsibilities:

Store embeddings

Maintain embedding index

Support fast similarity search

Tools Used:

ChromaDB

1.5 Retrieval Module

The Retrieval Module retrieves relevant document chunks based on user queries.

It compares query embeddings with stored embeddings to find the most relevant matches.

Primary Responsibilities:

Convert user query into embedding

Search vector database

Retrieve top relevant chunks

Tools Used:

LangChain Retriever

1.6 Query Processing Module

The Query Processing Module processes user input and prepares it for response generation.

This module combines retrieved chunks with user queries and sends them to the language model.

Primary Responsibilities:

Accept user query

Combine the query with the retrieved context

Prepare prompt for LLM

Tools Used:

Prompt Engineering

1.7 Graph Execution Module

The Graph Execution Module controls the workflow logic using LangGraph.

It defines how the system transitions between processing stages and handles conditional routing.

Primary Responsibilities:

Execute workflow nodes

Manage state transitions

Route data between modules

Tools Used:

LangGraph

1.8 HITL Module

The Human-in-the-Loop (HITL) Module manages escalation scenarios.

If the system fails to generate a confident response, the query is redirected to a human operator.

Primary Responsibilities:

Detect low-confidence responses

Escalate queries

Receive human input

Deliver the final response to the user

Tools Used:

Manual input interface

---

2. Data Structures

Data structures define how information is stored and transferred between different modules in the RAG-Based Customer Support Assistant system. Proper data structure design ensures efficient data handling, faster retrieval, and smooth workflow execution.

This section describes the key data structures used in the system.

2.1 Document Representation

The document representation structure stores information extracted from the input PDF file.

After loading the PDF, the text is organized into a structured format that includes page-level content. This structure allows the system to track the source of each text segment.

Structure Example:

```
Document {
    document_id: String
    page_number: Integer
    text_content: String
}
```

Description:

document_id uniquely identifies the document

page_number indicates the page from which text was extracted

text_content stores extracted text

This structure ensures that document content is organized and traceable.

2.2 Chunk Format

The chunk format defines how smaller segments of text are stored after chunking.

Each chunk contains a portion of the original document text. Chunking allows large documents to be divided into manageable sections for embedding and retrieval.

Structure Example:

```
Chunk {
    chunk_id: String
    document_id: String
    chunk_text: String
}
```

Description:

chunk_id uniquely identifies each chunk

document_id links chunk to source document

chunk_text contains actual text segment

This structure supports efficient chunk storage and retrieval.

2.3 Embedding Structure

The embedding structure stores numerical vector representations of chunks.

Each chunk is converted into an embedding vector, which is stored along with its corresponding chunk identifier.

Structure Example:

```
Embedding {
    embedding_id: String
    chunk_id: String
    vector: Float Array
}
```

Description:

embedding_id uniquely identifies the embedding

chunk_id connects the embedding to its chunk

A vector contains a numerical representation of text

This structure enables semantic search functionality.

2.4 Query-Response Schema

The Query-Response schema defines how user queries and system responses are stored.

It captures user input, retrieves context, and generates output.

Structure Example:

```
QueryResponse {
    query_id: String
    user_query: String
    retrieved_chunks: List
    generated_response: String
}
```

Description:

query_id uniquely identifies each query

user_query stores the input question

retrieved_chunks stores matching text segments

generated_response stores final output

This structure ensures proper tracking of system interactions.

2.5 Graph State Object

The Graph State Object stores workflow data that moves between LangGraph nodes.

It maintains the system state during query execution.

Structure Example:

```
GraphState {
   user_query: String
   retrieved_context: List
   response: String
   escalation_flag: Boolean
}
```

Description:

user_query stores the original question

retrieved_context stores relevant chunks

response stores generated an answer

escalation_flag determines HITL routing

This structure supports workflow management.

---

3. Workflow Design (LangGraph)

The workflow design defines how the RAG-Based Customer Support Assistant processes user queries using a graph-based workflow. LangGraph is used to control the execution flow between different processing steps.

In this system, the workflow is structured using nodes and edges, where each node represents a processing step and edges define transitions between steps.

The workflow ensures that user queries move through retrieval, response generation, and escalation stages in an organized manner.

3.1 Workflow Nodes

Nodes represent individual processing steps in the system. Each node performs a specific function during query processing.

The following nodes are used in the system.

Node 1: Input Processing Node

The Input Processing Node receives the user query and prepares it for retrieval.

Responsibilities:

Accept user query

Clean input text

Convert the query into an embedding

Send query embedding to the retrieval module

This node ensures that the user input is properly formatted before retrieval begins.

Node 2: Retrieval Node

The Retrieval Node retrieves relevant document chunks from the vector database.

Responsibilities:

Search vector database

Identify relevant chunks

Select top matching results

Send retrieved context to response generation node

This node ensures accurate context selection.

Node 3: Response Generation Node

The Response Generation Node generates an answer using the retrieved context and user query.

Responsibilities:

Combine the query with the retrieved context

Send prompt to language model

Generate response text

Pass response to evaluation node

This node produces the main output of the system.

Node 4: Evaluation Node

The Evaluation Node determines whether the generated response is reliable.

Responsibilities:

Check response confidence

Evaluate completeness

Determine escalation requirement

If confidence is low, escalation is triggered.

Node 5: Output Node

The Output Node sends the final response to the user.

Responsibilities:

Deliver the generated response

Display output

Log response details

This node completes the workflow.

3.2 Workflow Edges

Edges define how data flows between nodes.

The workflow edges in this system are:

Input Node → Retrieval Node

Retrieval Node → Response Generation Node

Response Generation Node → Evaluation Node

Evaluation Node → Output Node

Evaluation Node → HITL Node (if escalation required)

These edges ensure smooth movement of data between processing stages.

3.3 Workflow State

The workflow state stores data that moves between nodes.

Each node reads and updates the state during execution.

State Example:

Plain text

```
State {
   user_query: String
   retrieved_chunks: List
   response: String
   escalation_flag: Boolean
}
```

Explanation:

user_query contains input question

retrieved_chunks stores retrieved context

Response stores generated answer

escalation_flag controls the routing decision

3.4 Workflow Execution Flow

The complete workflow execution follows a structured sequence.

Execution Steps:

User enters query

Query moves to the Input Processing Node

Retrieval Node fetches relevant chunks

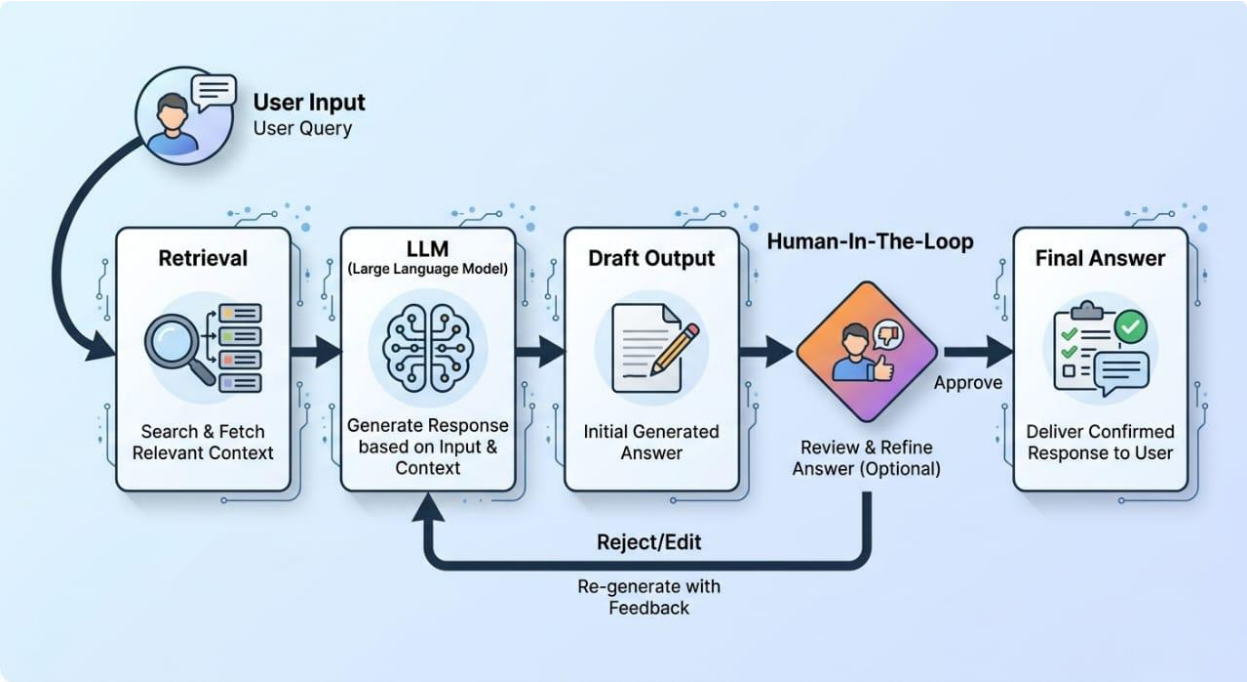
Response Generation Node generates an answer

Evaluation Node checks response quality

If response is valid → Output Node

If response is invalid → HITL Node

This structured flow ensures reliable and efficient response generation.



---

4. Conditional Routing Logic

Conditional routing logic defines how the system decides the next step based on the output generated during query processing. This logic allows the workflow to dynamically choose whether to generate a response or escalate the query to a human operator.

In the RAG-Based Customer Support Assistant, routing decisions are based on response quality, relevance, and confidence level.

This mechanism ensures that users receive accurate responses while minimizing incorrect outputs.

4.1 Answer Generation Criteria

The system generates an answer when sufficient relevant information is available from the retrieved document chunks.

During retrieval, the system selects the top matching chunks based on similarity scores. If the retrieved content is relevant and complete, the language model generates a response using that context.

Conditions for Answer Generation:

Relevant chunks are successfully retrieved

Retrieved context matches user query

Response generated by the language model is complete

Confidence level meets threshold value

If all conditions are satisfied, the system proceeds to generate the final output.

4.2 Escalation Criteria

Escalation occurs when the system cannot confidently answer the user query.

In such situations, the system routes the query to the Human-in-the-Loop (HITL) module for manual handling.

Escalation ensures that incorrect or incomplete responses are avoided.

Conditions for Escalation:

Low confidence score from the language model

No relevant chunks retrieved

Retrieved content is incomplete

Query is complex or outside document scope

This routing decision protects system reliability.

4.3 Low Confidence Detection

Low confidence detection is used to evaluate the reliability of generated responses.

Confidence can be estimated using similarity scores or evaluation logic.

Example Indicators:

Similarity score below threshold

Weak match between query and retrieved chunks

Short or incomplete generated response

Example Threshold:

Plain text

Confidence Score < 0.60 → Escalation Triggered

This mechanism ensures that uncertain responses are handled carefully.

4.4 Missing Context Detection

Missing context detection identifies cases where relevant information is not available in the knowledge base.

If the system cannot find suitable matching chunks, it avoids generating incorrect answers.

Detection Conditions:

No chunks retrieved

Retrieved chunks unrelated to query

Empty retrieval results

If missing context is detected, escalation is triggered.

4.5 Complex Query Detection

Some queries require advanced reasoning or multiple-step analysis.

If the system identifies complex queries beyond its capability, it routes them to the HITL module.

Examples of Complex Queries:

Multi-part questions

Ambiguous requests

Queries requiring external knowledge

Complex query detection improves response accuracy.

4.6 Routing Decision Flow

The routing process follows a structured decision sequence.

Routing Steps:

Retrieve relevant chunks

Generate response

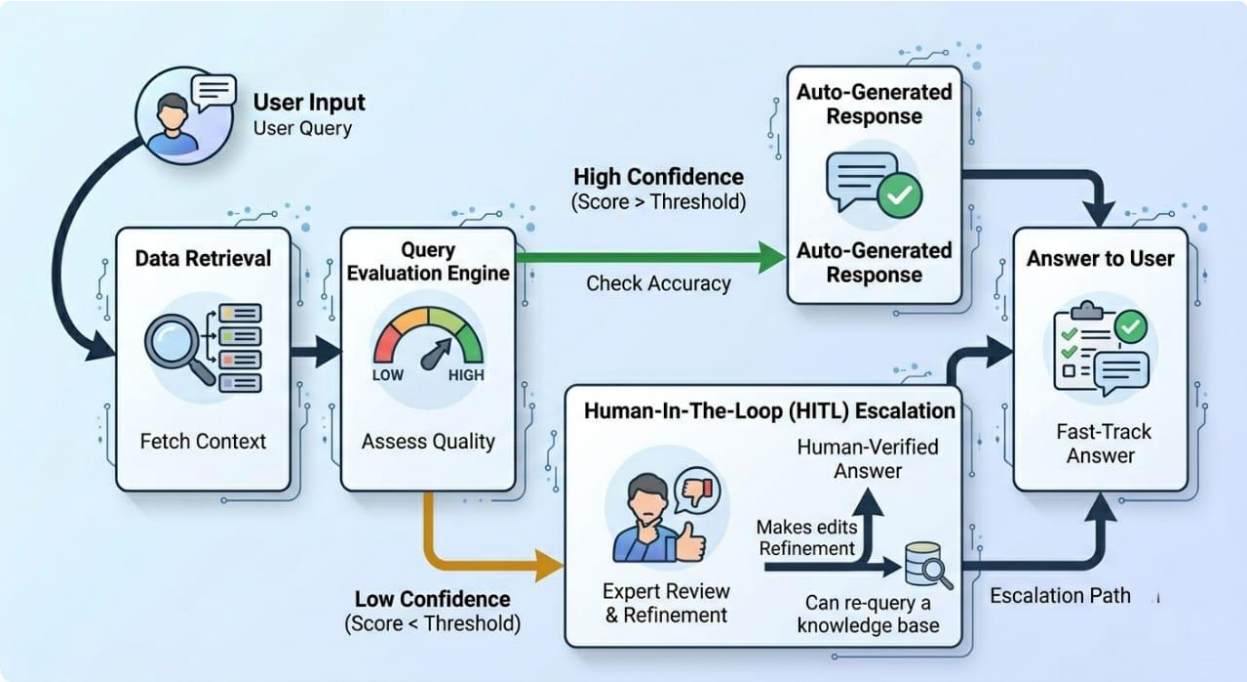
Evaluate response confidence

If confidence is high → Send response to user

If confidence is low → Escalate to HITL

If no chunks retrieved → Escalate immediately

This routing workflow ensures consistent system behavior.



5. HITL Design

The Human-in-the-Loop (HITL) design defines how the system integrates human intervention when automated responses are insufficient or unreliable.

HITL ensures that users receive accurate responses even when the system cannot confidently generate answers. This improves trust, reliability, and overall system effectiveness.

In the RAG-Based Customer Support Assistant, HITL is triggered when the system detects uncertainty or missing information.

5.1 Escalation Trigger Conditions

Escalation to human support occurs when the system fails to generate reliable responses.

These conditions are evaluated during the response evaluation stage.

Escalation is triggered when:

Confidence score is below threshold

No relevant document chunks are retrieved

Retrieved content is incomplete

Query is too complex

Generated response is unclear

These conditions ensure that unreliable responses are not delivered to users.

5.2 Human Interaction Workflow

When escalation occurs, the system forwards the query to a human operator.

The human operator reviews the query and provides a suitable response based on their understanding of the problem.

Human Workflow Steps:

System detects low-confidence response

Query is forwarded to human operator

Human reviews query

Human provides correct response

Response is sent back to user

This workflow ensures continuity in response handling.

5.3 Human Response Integration

After receiving the human-generated response, the system integrates it into the workflow.

The response is treated as the final output and delivered to the user.

Integration Steps:

Receive human response

Attach response to workflow state

Send response to output module

Log human-assisted interaction

This allows smooth transition from automated to manual processing.

5.4 Benefits of HITL Integration

The inclusion of human intervention provides several benefits.

Key Benefits:

Improves response accuracy

Prevents incorrect information delivery

Handles complex queries

Enhances user satisfaction

Builds trust in system responses

HITL ensures that critical queries receive reliable responses.

5.5 Limitations of HITL

Although HITL improves system reliability, it also introduces certain limitations.

Key Limitations:

Requires human availability

May increase response time

Additional operational cost

Manual intervention dependency

Despite these limitations, HITL significantly enhances system reliability.

---

6. API / Interface Design

The API and interface design define how users interact with the RAG-Based Customer Support Assistant system. This includes how queries are received, processed, and how responses are returned to the user.

In this project, the system is designed using a simple user interface such as a Command Line Interface (CLI) or web-based interface.

The interface ensures smooth communication between users and the backend workflow.

6.1 Input Format

The input format defines how user queries are submitted to the system.

Users provide queries in text format related to customer support issues. The system accepts natural language questions and processes them through the RAG workflow.

Input Structure Example:

Plain text

User Input:

"How can I reset my account password?"

Input Parameters:

user_query → Text-based customer question

timestamp → Time of query submission

user_id → Unique identifier for user (optional)

This format ensures that user queries are clearly captured.

6.2 Output Format

The output format defines how responses are returned to the user.

After processing the query, the system generates a structured response containing the final answer.

Output Structure Example:

System Output:

```
{
  "response": "To reset your password, click on 'Forgot Password' and follow the instructions.",
  "source": "Customer Support Guide",
  "escalation": False
}
```

Output Parameters:

response → Generated answer

source → Retrieved document reference

escalation → Indicates HITL usage

This structure ensures transparency in response generation.

6.3 Interaction Flow

The interaction flow describes the sequence of communication between the user and the system.

Interaction Steps:

User enters query

Query is sent to the system interface

System processes the query

Relevant information is retrieved

Response is generated

Response is displayed to the user

If required, escalation is triggered

This structured interaction improves usability.

6.4 Interface Type Selection

The system supports multiple interface types depending on deployment requirements.

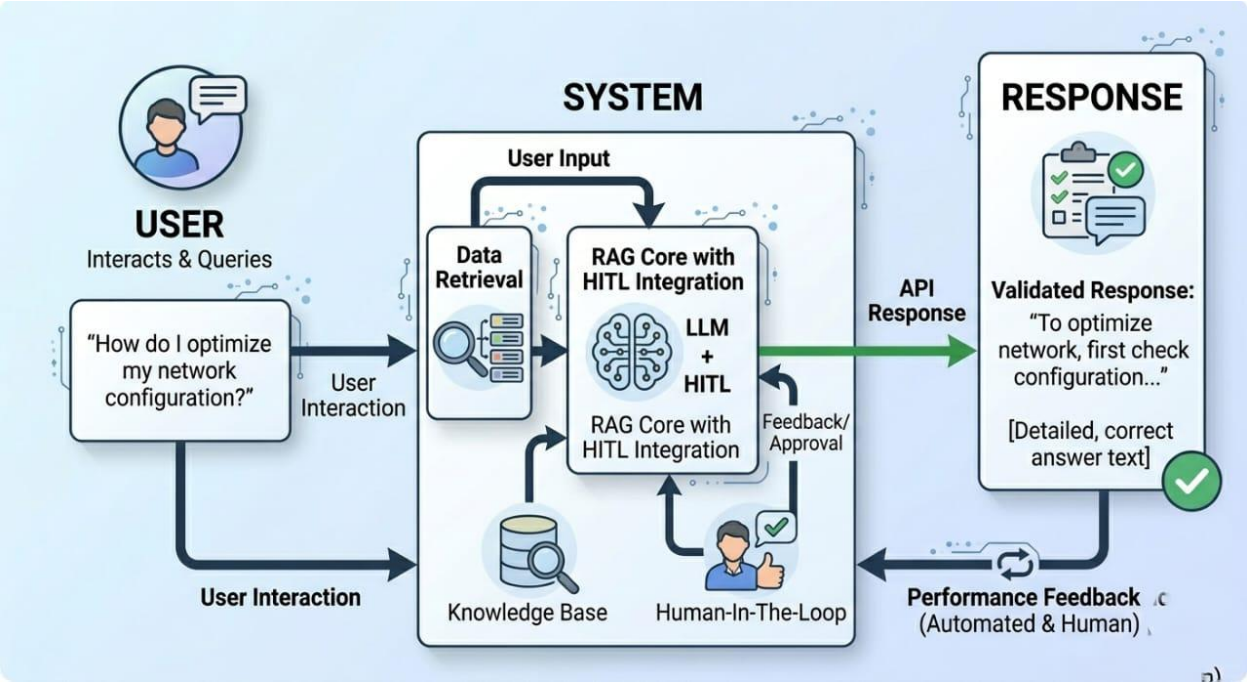
Possible Interfaces:

Command Line Interface (CLI)

Web-based interface

Chat-based interface

For this project, a CLI-based interface is sufficient because it allows quick testing and demonstration of system functionality.



7. Error Handling

Error handling is an essential component of the RAG-Based Customer Support Assistant system. It ensures that unexpected failures or missing information do not cause system crashes or incorrect responses.

Proper error handling improves system reliability, user experience, and fault tolerance.

This section describes common error scenarios and the strategies used to handle them.

7.1 Missing Document Error

A missing document error occurs when the system fails to locate or load the input PDF file.

This situation may occur if the file path is incorrect or if the file is deleted.

Handling Strategy:

Verify the file path before loading

Display error message to user

Request the user to upload a valid document

Prevent system crash

Example Message:

Error: Knowledge base document not found. Please upload a valid PDF file.

This ensures that the system handles missing files safely.

7.2 No Relevant Chunks Found

This error occurs when the retrieval system fails to find relevant chunks matching the user query.

This situation indicates a missing context in the knowledge base.

Handling Strategy:

Trigger escalation to HITL

Inform the user about missing information

Log query for future improvement

Example Message:

No relevant information found. Escalating to human support.

This prevents the generation of incorrect responses.

7.3 Embedding Generation Failure

Embedding generation may fail due to model errors or invalid text input.

Handling Strategy:

Retry embedding generation

Validate text before processing

Log failure details

Notify administrator if failure persists

Example Message:

Embedding generation failed. Retrying process.

This ensures system stability during vector creation.

7.4 LLM Response Failure

This error occurs when the language model fails to generate a response.

Possible reasons include network issues or model timeout.

Handling Strategy:

Retry response generation

Use a fallback message

Escalate to HITL if the retry fails

Example Message:

Response generation failed. Redirecting to human support.

This maintains response continuity.

7.5 Invalid User Input

Invalid input occurs when the user submits empty or meaningless queries.

Handling Strategy:

Validate user input

Reject empty queries

Prompt the user to enter a valid question

Example Message:

Plain text

Invalid input detected. Please enter a valid query.

This improves input quality.

7.6 System Logging and Monitoring

System logging records errors and system activity for debugging and maintenance.

Logging helps identify recurring issues and improve performance.

Logging Strategy:

Record all system errors

Maintain log files

Track system performance metrics

Monitor failure patterns

This supports long-term system maintenance.